Section 3.2 Source Code Documentation

RoamReady Web Application

Contributor: Muhtarim Afnan Chowdhury (mc802595)

Repository: TempRoam/RoamReady | Branch: main

Module and Class Header Summaries

Module and file	Date / programmer	Description and important functions	Inputs, outputs, data structures, and algorithm
Supabase client src/lib/supabase.ts	2026-10-01 mc802595 Commit 0c948ab	Creates and exports the shared browser Supabase client. The module reads the public project URL and publishable key from environment variables.	Inputs: NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY. Output: reusable Supabase client. Uses a single exported client so components share one configured connection.
Browser SSR client src/utils/supabase/client.ts	2026-11-10 mc802595 Commit a62383c	createClient() creates a browser client through createBrowserClient() for client-side authentication and database calls.	Input: none; configuration comes from public environment variables. Output: Supabase browser client. The factory approach keeps browser configuration in one utility module.
Server SSR client src/utils/supabase/server.ts	2026-11-10 mc802595 Commit a62383c	createClient(cookieStore) creates a server-side Supabase client and adapts Next.js cookie getAll/setAll handlers.	Input: Next.js cookie store. Output: server Supabase client. Cookie name/value/options objects preserve the session across server requests. The sequential cookie-adapter algorithm was selected for compatibility with Supabase SSR.
Google OAuth action src/app/auth/actions.ts	2026-11-10 mc802595 Commit a62383c	signInWithGoogle() starts Google authentication from a server action and redirects the browser to the provider URL.	Inputs: request headers and cookies. Output: redirect to Google or an encoded error. Header-derived origin handling supports local and deployed environments; redirect-based OAuth was selected because the provider completes authentication outside the application.
Auth callback src/app/auth/callback/route.ts	2026-11-10 mc802595 Commit a62383c	GET(request) completes the OAuth callback and exchanges the authorization code for a Supabase session.	Input: Request containing code and next URL parameters. Output: redirect to the requested page or login with an error. Uses a conditional success/error branch; code exchange was selected instead of trusting client-provided session data.
LoginPage src/app/login/page.tsx	2026-11-10 mc802595 Commit a62383c	Provides email/password login, Google login, password-reset navigation, and feedback messages. handleLogin(event) submits credentials with signInWithPassword(); handleGoogleLogin() starts OAuth.	Inputs: FormEvent, email, and password strings. Outputs: Promise<void>, authentication state, redirect, or error message. React state stores form values and feedback.

			Event-driven async submission was selected to avoid a full-page reload.
SignupPage src/app/signup/page.tsx	2026-11-10 mc802595 Commit a62383c	Creates accounts with full name, email, and password and supports Google signup. handleSignup(event) calls signUp(); handleGoogleSignup() calls signInWithOAuth().	Inputs: FormEvent, fullName, email, and password. Outputs: Promise<void> and success/error feedback. User metadata stores full_name. Supabase validation was selected instead of duplicating authentication rules in the frontend.
ResetPasswordPage src/app/reset-password/page.tsx	2026-11-10 mc802595 Commit a62383c	Collects an email address and submits a password-reset request to the server action.	Input: FormData containing email. Output: redirect with confirmation or error message. The form/server-action design keeps the request flow separate from the visual page and avoids exposing server-only logic in the client.
UpdatePasswordPage src/app/reset-password/update/page.tsx	2026-11-10 mc802595 Commit a62383c	Allows a recovery session to set a new password. handleUpdate(event) compares both password fields and calls updateUser({ password }).	Inputs: FormEvent, password, and confirmPassword strings. Output: Promise<void> and status message. A direct equality comparison prevents mismatched values before the database update; client-side validation gives immediate feedback.
NotificationsPage src/app/notifications/page.tsx	2026-11-10 mc802595 Commit a62383c	Loads notifications for the signed-in user and marks unread notifications as read. loadNotifications() gets the user and queries the table; markAsRead(notificationId) updates one row.	Notification contains id, title, message, type, is_read, and created_at. Inputs: notification ID and authenticated user. Output: ordered notification array and updated read state. The query sorts newest first; map() immutably updates local state after the database update.
Navbar src/components/layout/Navbar.tsx	2026-11-10 mc802595 Commit a62383c	Provides session-aware navigation and logout. checkUser() reads the current user; handleLogout() calls signOut() and redirects; onAuthStateChange() keeps the UI synchronized.	Input: Supabase session events. Output: login/signup or logout navigation. Boolean isLoggedIn state represents session status and the auth subscription is cleaned up on unmount. Event subscription was selected so navigation updates without manual page refreshes.
Database security migration supabase/migrations/20261009021656_initial_schema.sql	2026-10-09 mc802595 Security-policy contribution	Defines the initial profiles/trips database structures and Row Level Security policies used by the application.	Inputs: SQL migration execution and auth.uid(). Outputs: tables, foreign keys, RLS rules, and policies. Policies compare the authenticated user to the record owner, enforcing authorization at the database layer rather than relying only on frontend checks.

Algorithms and Design Decisions

- Authentication uses asynchronous, event-driven form handlers. preventDefault() avoids an unnecessary page reload while Supabase returns a success or error response.

- OAuth uses an authorization-code callback. The callback exchanges the code for a session so the server can establish authenticated state without trusting a client-supplied session object.
- Password recovery validates that both password fields match before calling updateUser(). This prevents an avoidable network request and gives immediate user feedback.
- Notifications are ordered by created_at descending so the newest alerts appear first. Array.map() updates only the selected notification in local React state.
- Row Level Security uses auth.uid() to enforce record ownership in the database. This is safer than relying only on UI restrictions because the rule applies to database requests directly.